

P4Prometheus Installation

Table of Contents

Deployment Roles	2
Automated Installation	2
Script Reference	3
Upgrades	3
Enterprise Options	3
Dedicated Data Volume	3
Retention, Binaries, and Targets	4
Air-Gapped Installation	4
Install State and Fleet Configuration	4
Verify Installation	4
Manual Installation	5
Package Installation of Grafana	5
Create your first Data Source	5
Setup of Grafana dashboards	5
Script to create Grafana dashboard	6
Install Prometheus	7
Prometheus config	8
Install victoria metrics (optional but recommended)	9
Substituting Victoria Metrics for Prometheus in Grafana	11
Importing Prometheus data into Victoria Metrics	11
Install node exporter	11
Install p4prometheus - details	13
Install p4metrics and monitor_locks systemd timer services	16
Checking for blocked commands	16
Start and enable service	17
Alerting	18
Alertmanager config	19
Alerting rules	19
Prometheus config to reference alertmanager rules	20
Troubleshooting	21
p4prometheus	21
p4metrics	22
node exporter	22
prometheus	23
Grafana	23
Advanced config options	23

Windows Installation	23
Windows Exporter	23
P4prometheus on Windows	24
Running p4metrics	24
Installing Programs as Services	24

P4Prometheus installation is automated by the scripts in [scripts/](#). Manual component installation is retained in [Manual Installation](#) for environments where automation cannot be used.

Deployment Roles

Install the following components for each role:

Role	Installer and components
Monitoring server	install_prom_graf.sh : Grafana, Prometheus, VictoriaMetrics, node_exporter, and optional Alertmanager, Pushgateway, and Pint.
Helix Core server	install_p4prom.sh : node_exporter, p4prometheus, p4metrics, and lock-monitoring scripts.
Other monitored server	install_node.sh : node_exporter only, for example on Swarm, Hansoft, or Helix TeamHub hosts.

Automated Installation

Run the relevant installer as **root**. Download the script from a checkout, or download its raw GitHub URL, make it executable, and review the available options before installing:

```
wget
https://raw.githubusercontent.com/perforce/p4prometheus/master/scripts/install_prom_graf.sh
chmod +x install_prom_graf.sh
./install_prom_graf.sh -h
```

For a Helix Core server, download [install_p4prom.sh](#) together with [p4prom_common.sh](#) into the same directory:

```
wget
https://raw.githubusercontent.com/perforce/p4prometheus/master/scripts/install_p4prom.sh
wget
https://raw.githubusercontent.com/perforce/p4prometheus/master/scripts/p4prom_common.sh
```

```
chmod +x install_p4prom.sh
./install_p4prom.sh -h

# SDP instance 1
./install_p4prom.sh 1
```

For a non-SDP server, use `./install_p4prom.sh -nosdp -h` to review the required connection and path options before installing.

Script Reference

- [install_prom_graf.sh](#): install the monitoring server.
- [update_prom_graf.sh](#): update the monitoring server.
- [install_p4prom.sh](#): install monitoring on a Helix Core server.
- [update_p4prom.sh](#): update monitoring on a Helix Core server.
- [install_node.sh](#): install node_exporter on another monitored server.
- [install_lslocks.sh](#): install lock monitoring only.
- [migrate_prom_graf_data.sh](#): move monitoring-server data to another volume.

Upgrades

In an SDP environment, run the update checker as the `perforce` user:

```
cd /p4/common/site/bin
./check_for_updates.sh
sudo ./update_p4prom.sh 1
```

For very old installations where `update_p4prom.sh` was not downloaded, remove `.update_config` and rerun `check_for_updates.sh`. Alternatively, download [update_p4prom.sh](#) directly, review `-h`, and run it as `root`.

Enterprise Options

All install and update scripts support a common set of options. Run the appropriate script with `-h` for the complete list.

Dedicated Data Volume

Use `-d` to keep monitoring data on a dedicated volume:

```
sudo ./install_prom_graf.sh -d /data
sudo ./update_prom_graf.sh
```

The selected path is retained in `/etc/p4prometheus-monitoring/install.env`. To move an existing installation, use:

```
sudo ./migrate_prom_graf_data.sh -d /data --dry-run
sudo ./migrate_prom_graf_data.sh -d /data
sudo ./migrate_prom_graf_data.sh -d /data --cleanup-old
```

Retention, Binaries, and Targets

```
# Retain metrics for 12 months and use a custom binary directory.
sudo ./install_prom_graf.sh -d /data -r 12 -b /opt/monitoring/bin

# Add scrape targets during installation.
sudo ./install_prom_graf.sh -target myp4server:9100 -target replica1:9100
```

Air-Gapped Installation

Download the required GitHub release assets on a connected machine, transfer them to the target, then pass their directory with `--local-tarballs-dir`:

```
sudo ./install_prom_graf.sh -d /data --local-tarballs-dir /opt/installers
sudo ./install_p4prom.sh --local-tarballs-dir /opt/installers
```

`p4prom_common.sh` must be alongside `install_p4prom.sh` or `update_p4prom.sh` on air-gapped Helix Core servers.

Install State and Fleet Configuration

Monitoring-server scripts save state in `/etc/p4prometheus-monitoring/install.env`. Helix Core server scripts use `/p4/common/site/config/p4prom_install.env` with SDP, or `/etc/p4prometheus/p4prom_install.env` without it.

For HMS fleets sharing `/p4/common`, create a hostname-specific configuration when required:

```
cp /p4/common/site/config/p4prometheus.yml \
  /p4/common/site/config/p4prometheus.$(hostname -s).yml
systemctl restart p4prometheus
```

The generated wrappers choose the hostname-specific file before the shared default.

Verify Installation

```
systemctl status p4prometheus p4metrics node_exporter
curl http://localhost:9100/metrics | grep p4_
```

On the monitoring server, verify Prometheus targets and dashboards after running the relevant installer. See [\[troubleshooting\]](#) for component-specific checks.

Manual Installation

The following component-by-component procedures are retained for sites that cannot use the scripts. They are not the recommended default installation path.

Package Installation of Grafana

This should be done on the monitoring server only. If not using [automated scripts](#) then follow these instructions.

Use the appropriate link below depending if you using **apt** or **yum**:

- <https://grafana.com/docs/grafana/latest/installation/debian/>
- <https://grafana.com/docs/grafana/latest/installation/rpm/>

Create your first Data Source

Assuming you are using Victoria Metrics (VM):

- Data source > new
 - Type is Prometheus (VM is API compatible)
 - Name is **Victoria Metrics**
 - Target/port: <http://localhost:8428>
- Click **Save and test**

Setup of Grafana dashboards

Once Grafana is installed (and Prometheus/Victoria Metrics) the following dashboards are recommended to be imported as starting points for further customisation:

- <https://grafana.com/grafana/dashboards/12278> - P4 Stats
- <https://grafana.com/grafana/dashboards/15509> - P4 Stats (non-SDP)
- <https://grafana.com/grafana/dashboards/405> - Node Exporter Server Info
- <https://grafana.com/grafana/dashboards/1860> - Node Exporter Full
- <https://grafana.com/grafana/dashboards?search=node%20exporter>

They can be imported from Grafana dashboard management page.

- Dashboard > New > Import

Alternatively see below for experimental script to create dashboards which is easier to customize.

See an example of [interpreting Linux prometheus performance metrics](#)

For Windows see [Windows Installation](#) since Windows Exporter is used instead of Node Exporter.

If first time with Grafana, the default user/pwd: **admin/admin**. It will prompt you to create a proper password!

Script to create Grafana dashboard

Download the following files:

- [create_dashboard.py](#) or for use with wget, download raw file: [right click this link > copy link address](#)
- [dashboard.yaml](#) or for use with wget, download raw file: [right click this link > copy link address](#)

Create a Grafana service account token (or legacy API key token) for your Grafana installation.

Version notes (dashboard generation/upload):

- Tested with Grafana 9.x, 10.x, 11.x, and 12.x using the classic dashboard API endpoint **/api/dashboards/db**.
- Recommended Python package versions: **grafanalib>=0.7,<1.0** and **requests>=2.28**.
- The script accepts both modern service account tokens (**GRAFANA_API_TOKEN**) and legacy API keys (**GRAFANA_API_KEY**).

Install Python dependencies (recommended: uv):

```
curl -LsSf https://astral.sh/uv/install.sh | sh
uv venv .venv
source .venv/bin/activate
uv pip install "grafanalib>=0.7,<1.0" "requests>=2.28"
```

Fallback (pip):

```
pip3 install grafanalib requests
```

Optional: verify installed package versions

```
python3 -c "import grafanalib, requests; print('grafanalib', getattr(grafanalib, '__version__', 'unknown')); print('requests', requests.__version__)"
```

Set environment variables:

```
export GRAFANA_SERVER=https://p4monitor:3000
export GRAFANA_API_TOKEN="<service account token created above>"
```

or legacy variable name:

```
export GRAFANA_API_KEY="<API key created above>"
```

Review the `dashboard.yaml` file and adjust to your local site names where appropriate.

```
vi dashboard.yaml
```

Create and upload the dashboard:

```
./create_dashboard.py --title "P4Prometheus" --url $GRAFANA_SERVER --api-key
$GRAFANA_API_KEY
```

You can re-upload the dashboard with the same title (it will create a new version).

Install Prometheus

This must be done on the monitoring server only. If not using [automated scripts](#) then follow these instructions.

IMPORTANT

Once you have installed prometheus you will need to adjust its configuration file!

Run the following as root:

```
sudo useradd --no-create-home --shell /bin/false prometheus
sudo mkdir /etc/prometheus
sudo mkdir /var/lib/prometheus
sudo chown prometheus:prometheus /etc/prometheus
sudo chown prometheus:prometheus /var/lib/prometheus

export PVER="2.33.5"
wget https://github.com/prometheus/prometheus/releases/download/v$PVER/prometheus-
$PVER.linux-amd64.tar.gz

tar xvf prometheus-$PVER.linux-amd64.tar.gz
mv prometheus-$PVER.linux-amd64 prometheus-files

sudo cp prometheus-files/prometheus /usr/local/bin/
sudo cp prometheus-files/promtool /usr/local/bin/
sudo chown prometheus:prometheus /usr/local/bin/prometheus
```

```

sudo chown prometheus:prometheus /usr/local/bin/promtool
sudo chmod 755 /usr/local/bin/prometheus
sudo chmod 755 /usr/local/bin/promtool

sudo cp -r prometheus-files/conssoles /etc/prometheus
sudo cp -r prometheus-files/console_libraries /etc/prometheus
sudo chown -R prometheus:prometheus /etc/prometheus/conssoles
sudo chown -R prometheus:prometheus /etc/prometheus/console_libraries

```

Create service file:

```

cat << EOF > /etc/systemd/system/prometheus.service
[Unit]
Description=Prometheus
Wants=network-online.target
After=network-online.target

[Service]
User=prometheus
Group=prometheus
Type=simple
ExecStart=/usr/local/bin/prometheus \
    --config.file /etc/prometheus/prometheus.yml \
    --storage.tsdb.path /var/lib/prometheus/ \
    --web.console.templates=/etc/prometheus/conssoles \
    --web.console.libraries=/etc/prometheus/console_libraries

[Install]
WantedBy=multi-user.target
EOF

```

Prometheus config

It is important you edit and adjust the `targets` value appropriately below (see `#####` section) to scrape from your commit/edge/replica servers (and localhost).

See later section on enabling Alertmanager if required.

```

cat << EOF > /etc/prometheus/prometheus.yml
global:
  scrape_interval:     15s # Set the scrape interval to every 15 seconds. Default is
                           every 1 minute.
  evaluation_interval: 15s # Evaluate rules every 15 seconds. The default is every 1
                           minute.
  # scrape_timeout is set to the global default (10s).

# Alertmanager configuration - optional - uncomment if using
# alerting:

```

```
# alertmanagers:
# - static_configs:
#   - targets:
#     - localhost:9093

# Load rules once and periodically evaluate them according to the global
'evaluation_interval'.
# Uncomment if using alertmanager
# rule_files:
# - "perforce_rules.yml"

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'node_exporter'
    static_configs:
      #####
      # CONFIGURE THESE VALUES AS APPROPRIATE FOR YOUR SERVERS!!!!
      # Note that the names here will appear as labels in your metrics.
      # Recommend not using IP address as not user friendly!
      # The port is going to be 9100 by default for node_exporter unless using Windows
      Exporter targets
      #####
      - targets:
        - p4hms:9100
        - p4main:9100
        - p4_ha:9100

EOF
```

Make sure user has access:

```
sudo chown prometheus:prometheus /etc/prometheus/prometheus.yml
```

Install victoria metrics (optional but recommended)

This is a high performing component (up to 20x faster) and good for long term storage (data compression is up to 70x) so that much more data can be stored in the same space. If not using [automated scripts](#) then follow these instructions.

It is API compatible and thus a drop in for querying. It is configured as a Prometheus writer so is continually kept up-to-date.

Run the following as root:

```

export PVER="1.87.5"    # Adjust to suitable recent value!
wget
https://github.com/VictoriaMetrics/VictoriaMetrics/releases/download/v$PVER/victoria-
metrics-v$PVER.tar.gz
wget
https://github.com/VictoriaMetrics/VictoriaMetrics/releases/download/v$PVER/vmutils-
v$PVER.tar.gz

tar zxvf victoria-metrics-v$PVER.tar.gz
tar zxvf vmutils-v$PVER.tar.gz

mv victoria-metrics-prod /usr/local/bin/
mv vmagent-prod /usr/local/bin/
mv vmalert-prod /usr/local/bin/
mv vmauth-prod /usr/local/bin/
mv vmbackup-prod /usr/local/bin/
mv vmrestore-prod /usr/local/bin/

```

Create service file (**adjust retentionPeriod as desired**):

```

cat << EOF > /etc/systemd/system/victoria-metrics.service
[Unit]
Description=Victoria Metrics
Wants=network-online.target
After=network-online.target

[Service]
User=prometheus
Group=prometheus
Type=simple
ExecStart=/usr/local/bin/victoria-metrics-prod \
    -storageDataPath /var/lib/victoria-metrics/ \
    -retentionPeriod=6

[Install]
WantedBy=multi-user.target
EOF

```

Consider adjusting the **retentionPeriod** value in the config file.

Ensure data directory exists and is properly owned:

```

sudo mkdir /var/lib/victoria-metrics
sudo chown -R prometheus:prometheus /var/lib/victoria-metrics

```

Start and enable service:

```
sudo systemctl daemon-reload
sudo systemctl enable victoria-metrics
sudo systemctl start victoria-metrics
sudo systemctl status victoria-metrics
```

Check logs for service in case of errors:

```
journalctl -u victoria-metrics --no-pager | tail
```

Update the Prometheus config file `/etc/prometheus/prometheus.yml`, by adding the following section to the end of the file (it starts in the first column):

```
remote_write:
  - url: http://localhost:8428/api/v1/write
```

Note in the above it is possible to customize the port - but usually not required.

Either start or restart Prometheus:

```
sudo systemctl restart prometheus
make restart # Convenience command
```

Substituting Victoria Metrics for Prometheus in Grafana

If using Victoria Metrics, then you should:

- Create a suitable data source in Grafana (e.g. use the Prometheus type since VM is API compatible, but specify target as <http://localhost:8428>)
- Click the **Test and Save** button
- Change existing dashboards to use it instead of Prometheus (it is API compatible)

Importing Prometheus data into Victoria Metrics

This can be fairly easily done, and will allow you to save the space used by Prometheus.

See [taking a snapshot via web api](#)

Then import it [using vmctl](#)

Install node exporter

This must be done on the Perforce (Helix Core) server machine (ditto for any other servers such as replicas which are being monitored).

Run the following as root:

```
useradd --no-create-home --shell /bin/false node_exporter

export PVER="1.3.1"
wget
https://github.com/prometheus/node_exporter/releases/download/v$PVER/node_exporter-
$PVER.linux-amd64.tar.gz

tar xvf node_exporter-$PVER.linux-amd64.tar.gz
mv node_exporter-$PVER.linux-amd64/node_exporter /usr/local/bin/
```

If you are installing on a Helix Core commit or replica server, then create a metrics directory, give ownership to account writing metrics, and make sure it has global read access (so `node_exporter` account can read entries)

```
mkdir /hxlogs/metrics
chown perforce:perforce /hxlogs/metrics
ls -al /hxlogs/metrics
```

Ensure the above has global read access (e.g. user `perforce` will write files, user `node_exporter` will read them).

If using SDP, then suggest this:

```
ln -s /hxlogs/metrics /p4/
chown -h perforce:perforce /p4/metrics
```

Create service file (note value for directory - may need to be adjusted).

Note that when installing on the monitoring machine, you do not need the `/hxlogs/metrics` directory, and you don't need the `collector.textfile.directory` parameter in the service file shown below.

```
cat << EOF > /etc/systemd/system/node_exporter.service
[Unit]
Description=Node Exporter
Wants=network-online.target
After=network-online.target

[Service]
User=node_exporter
Group=node_exporter
Type=simple
ExecStart=/usr/local/bin/node_exporter --collector.textfile.directory=/hxlogs/metrics

[Install]
```

```
WantedBy=multi-user.target
EOF
```

If using *Alertmanager* and wanting to check on the status of systemd services, the **ExecStart** line above should be:

```
ExecStart=/usr/local/bin/node_exporter --collector.systemd \
  --collector.systemd.unit-include=(p4.*|node_exporter)\.service \
  --collector.textfile.directory=/hxlogs/metrics
```

Check the **node_exporter** help, but the above will collect info on the specified services only.

Start and enable service:

```
sudo systemctl daemon-reload
sudo systemctl enable node_exporter
sudo systemctl start node_exporter
sudo systemctl status node_exporter
```

Check logs for service in case of errors:

```
journalctl -u node_exporter --no-pager | tail
```

Check that metrics are being exposed:

```
curl http://localhost:9100/metrics | less
```

Install p4prometheus - details

This must be done on the Perforce (Helix Core) server machine (and any replica machines).

This assumes SDP structure is in use on the server, and thus that user **perforce** exists.

Get latest release download link: <https://github.com/perforce/p4prometheus/releases>

Run the following as **root** (using link copied from above page):

```
export PVER=0.9.6 # Adjust this to latest release!
wget
https://github.com/perforce/p4prometheus/releases/download/v$PVER/p4prometheus.linux-
amd64.gz
gunzip p4prometheus.linux-amd64.gz
chmod +rx p4prometheus.linux-amd64
mv p4prometheus.linux-amd64 /usr/local/bin/p4prometheus
```

As user **perforce** run as below.

Important to check configuration values, e.g. **log_path**, **metrics_output** etc.

```
cat << EOF > /p4/common/config/p4prometheus.yaml
# -----
# sdp_instance: SDP instance - typically integer, but can be
# See: https://swarm.workshop.perforce.com/projects/perforce-software-sdp for more
# If this value is blank then it is assumed to be a non-SDP instance.
sdp_instance: 1

# -----
# log_path: Path to p4d server log - REQUIRED!
# Recommended to set an absolute path, e.g. /p4/1/logs/log
log_path: /p4/1/logs/log

# -----
# metrics_output: Name of output file to write for processing by node_exporter.
# Ensure that node_exporter user has read access to this folder.
metrics_output: /hxlogs/metrics/p4_cmds.prom

# -----
# server_id: Optional - serverid for metrics.
# If SDP install then it will read /p4/<sdp_instance>/root/server.id for
# If non-SDP install, set this field or set server_id_path (this field has
# preference!)
server_id:

# -----
# server_id_path: Optional - path to server.id file for metrics - only used if non-SDP
# install.
# If non-SDP install, set either this field, or server_id instead.
# e.g. server_id_path: /opt/perforce/server/root/server.id
server_id_path:

# -----
# output_cmds_by_user: true/false - Whether to output metrics
# p4_cmd_user_counter/p4_cmd_user_cumulative_seconds
# Normally this should be set to true as the metrics are useful.
# If you have a p4d instance with thousands of users you may find the number
# of metrics labels is too great (one per distinct user), so set this to false.
output_cmds_by_user: true

# -----
# case_sensitive_server: true/false - if output_cmds_by_user=true then if this value
# is set to false
# all userids will be written in lowercase - otherwise as they occur in the log file
# If not present, this value will default to true on Windows and false otherwise.
case_sensitive_server: true
```

```
# -----
# output_cmds_by_ip: true/false - Whether to output metrics
# p4_cmd_ip_counter/p4_cmd_ip_cumulative_seconds
# Like output_cmds_by_user this can be an issue for larger sites so defaults to false.
output_cmds_by_ip: true

# -----
# output_cmds_by_user_regex: Specifies a Go regex for users for whom to output
# metrics p4_cmd_user_detail_counter (tracks cmd counts per user/per cmd) and
# p4_cmd_user_detail_cumulative_seconds
#
# This can be set to values such as: "" (no users), ".*" (all users), or
# "swarm|jenkins"
# for just those 2 users. The latter is likely to be appropriate in many sites (keep
# an eye
# on automation users only, without generating thousands of labels for all users)
output_cmds_by_user_regex: ""

EOF
```

chown perforce:perforce /p4/common/config/p4prometheus.yaml

As user **root**:

Create service file as below - parameters you may need to customise:

- **User**
- **Group**
- **ExecStart**

```
cat << EOF > /etc/systemd/system/p4prometheus.service
[Unit]
Description=P4prometheus
Wants=network-online.target
After=network-online.target

[Service]
User=perforce
Group=perforce
Type=simple
ExecStart=/usr/local/bin/p4prometheus --config=/p4/common/config/p4prometheus.yaml

[Install]
WantedBy=multi-user.target
EOF
```

Start and enable service:

```
sudo systemctl daemon-reload
sudo systemctl enable p4prometheus
sudo systemctl start p4prometheus
sudo systemctl status p4prometheus
```

Check logs for service in case of errors:

```
journalctl -u p4prometheus --no-pager | tail
```

Check that metrics are being written:

```
grep lines /hxlogs/metrics/p4_cmds.prom
```

Install p4metrics and monitor_locks systemd timer services

Use [Automated Script Installation!!](#)

Check out: [install_p4metrics function in installer script for details](#)

Note that **p4metrics** has now replaced **monitor_metrics.sh** - the latter is retained as an example, but should not be installed!

Please note that **monitor_metrics.py** (which is called by **monitor_wrapper.sh**) runs **lslocks** and cross references locks found with **p4 monitor show** output. This is incredibly useful for determining processes which are blocked by other processes. It is hard to discover this information if you are not collecting the data at the time!

Warning: make sure that **lslocks** is installed on your Linux distribution!

The **monitor_metrics.py** (which is called by **monitor_wrapper.sh**) were previously installed in **crontab**. They are now installed as a systemd timer service.

If not using SDP then please ensure that an appropriate LONG TERM TICKET is setup in the environment that this script is running in.

Checking for blocked commands

Look in the log file **/p4/1/logs/monitor_metrics.log** (or wherever you have configured it to go) for output.

e.g. the following will find all info messages

```
grep "blocked by pid" /p4/1/logs/monitor_metrics.log | less
```

Other options to remove client locks (which are output with paths like `.../server.locks/clients/90,d/<client name>`):

```
grep "blocked by pid" /p4/1/logs/monitor_metrics.log | grep -v /clients | less
```

Output might contain lines like:

```
2021-06-08 10:42:01 pid 4203, user bldagent, cmd client, table
/hxmetadata/p4/1/db1/db.have, blocked by pid 3877, user jim, cmd sync, args
c:\Users\jim\p4servers\...#have
```

It may also be worth checking for commands which are "blocking" others, e.g.

```
grep blocking /p4/1/logs/monitor_metrics.log

2024-04-04 00:29:02: blocking cmd: elapsed 02:09:23, pid 284460, user perforce, cmd
sync, blocking 2, indirectly 203
2024-04-04 00:29:02: blocking cmd: elapsed 00:59:35, pid 374896, user fred, cmd dm-
CommitSubmit, blocking 203, indirectly 0
2024-04-04 00:29:02: blocking cmd: elapsed 00:30:45, pid 396091, user fred, cmd
shelve, blocking 3, indirectly 0
```

Please note that metrics (counts of processes locked) are written to `/p4/metrics/locks.prom` (or your metrics dir) and will be available to Prometheus/Grafana. See [P4Prometheus Metrics \(look for p4_lock*\)](#).

Start and enable service

```
sudo systemctl daemon-reload
sudo systemctl start prometheus
sudo systemctl status prometheus
sudo systemctl enable prometheus
```

Check logs for service in case of errors:

```
journalctl -u prometheus --no-pager | tail
```

Check that prometheus web interface is running:

```
curl http://localhost:9090/
```

Or open URL in a browser.

Alerting

Done via **alertmanager**. Optional component.

Please see also the examples of [Runbook Alert Handling doc](#) which can be customised for your specific site/organisation.

Setup is very similar to the above.

```
sudo useradd --no-create-home --shell /bin/false alertmanager
sudo mkdir /etc/alertmanager
sudo mkdir /var/lib/alertmanager
sudo chown alertmanager:alertmanager /etc/alertmanager
sudo chown alertmanager:alertmanager /var/lib/alertmanager

export PVER="0.23.0"
wget https://github.com/prometheus/alertmanager/releases/download/v$PVER/alertmanager-$PVER.linux-amd64.tar.gz

tar xvf alertmanager-$PVER.linux-amd64.tar.gz
mv alertmanager-$PVER.linux-amd64 alertmanager-files

sudo cp alertmanager-files/alertmanager /usr/local/bin/
sudo cp alertmanager-files/amtool /usr/local/bin/
sudo chown alertmanager:alertmanager /usr/local/bin/alertmanager
sudo chown alertmanager:alertmanager /usr/local/bin/amtool
sudo chmod 755 /usr/local/bin/alertmanager
sudo chmod 755 /usr/local/bin/amtool
```

```
cat << EOF > /etc/systemd/system/alertmanager.service
[Unit]
Description=Alertmanager
Wants=network-online.target
After=network-online.target

[Service]
User=alertmanager
Group=alertmanager
Type=simple
ExecStart=/usr/local/bin/alertmanager --config.file=/etc/alertmanager/alertmanager.yml \
    --storage.path=/var/lib/alertmanager --log.level=debug

[Install]
WantedBy=multi-user.target
EOF
```

Start and enable service:

```
sudo systemctl daemon-reload
sudo systemctl enable alertmanager
sudo systemctl start alertmanager
sudo systemctl status alertmanager
```

Check logs for service in case of errors:

```
journalctl -u alertmanager --no-pager | tail
```

Alertmanager config

See sample config file here:

- [EXAMPLES.md](#)

Strongly recommend: set up a simple **Makefile** in **/etc/alertmanager** which validates config file:

Note that Makefile format requires a **<tab>** char (not spaces) at the start of 'action' lines.

```
# Makefile for alertmanager
validate:
    amtool check-config alertmanager.yml

restart: validate
    sudo systemctl restart alertmanager
    sudo systemctl restart prometheus
```

Then you can validate your config:

```
# make validate
amtool check-config alertmanager.yml
Checking 'alertmanager.yml' SUCCESS
Found:
- global config
- route
- 0 inhibit rules
- 1 receivers
- 0 templates
```

Alerting rules

This is an example, assuming simple email and local postfix or equivalent have been setup.

See the version in: [EXAMPLES.md](#)

It would be setup as **/etc/prometheus/perforce_rules.yml**

Split-file alerting rules (recommended)

`update_prom_graf.sh` manages two separate rules files to avoid overwriting your customizations on each update:

File	Managed by	Description
<code>/etc/prometheus/perforce_rules.yml</code>	Script (auto-updated)	Upstream rules from this project. Overwritten when upstream changes.
<code>/etc/prometheus/perforce_rules_local.yml</code>	You	Your site-specific rules. Never touched by the script. Created with helpful comments on first run.

On each `update_prom_graf.sh` run:

- If the upstream `perforce_rules.yml` has changed and you have not modified it locally, it is silently overwritten.
- If both the upstream has changed **and** you have local edits, the script backs up your version as `perforce_rules.yml.YYYYMMDD` before installing the new upstream, then prints a warning reminding you to review and merge any new upstream rules into your `perforce_rules_local.yml`.

To activate both files, reference them in `prometheus.yml`:

```
rule_files:
  - "perforce_rules.yml"
  - "perforce_rules_local.yml"
```

Then uncomment the relevant section in `prometheus.yml`:

```
# Alertmanager configuration - optional
alerting:
  alertmanagers:
    - static_configs:
        - targets:
            - localhost:9093

# Load rules once and periodically evaluate them according to the global
'evaluation_interval'.
rule_files:
  - "perforce_rules.yml"
```

Prometheus config to reference alertmanager rules

Strongly recommend: set up a simple `Makefile` in `/etc/prometheus` which validates config and rules

file:

Note that Makefile format requires a `<tab>` char (not spaces) at the start of 'action' lines.

```
# Makefile for prometheus - convenience for validating and restarting the service
validate:
    promtool check config prometheus.yml

# Always validates first before restarting
restart: validate
    systemctl restart prometheus
```

Then you can validate your config:

```
# make validate
promtool check config prometheus.yml
Checking prometheus.yml
    SUCCESS: 1 rule files found

Checking performe_rules.yml
    SUCCESS: 8 rules found
```

For an example of a good starter alertmanager rules file see: [example prometheus_rules.yml file](#)

See also the [example pint configuration](#) - this is a very valuable linter/validator for alerting rules!

Troubleshooting

Make sure all *firewalls* are appropriately configured and the various components on each machine can see each other!

Port defaults are:

- Grafana: 3000
- Prometheus: 9090
- Victoria Metrics: 8428
- Node_exporter: 9100
- Alertmanager: 9093

Use curl on the monitoring server to ensure that you can pull metrics from the other servers (from the target Node Exporter ports) - see below.

p4prometheus

If this is running correctly, it should write into the designated log file, e.g. `/hxlogs/metrics/p4_cmds.prom`

You can just grep for the most basic metric a couple of times (make sure it is increasing every minute or so):

```
$ grep lines /p4/metrics/p4_cmds.prom  
  
# HELP p4_prom_log_lines_read A count of log lines read  
# TYPE p4_prom_log_lines_read counter  
p4_prom_log_lines_read{serverid="master.1",sdpinst="1"} 7143
```

p4metrics

Make sure **p4metrics** is working:

```
sudo systemctl status p4metrics  
sudo journalctl -u p4metrics --no-pager | less
```

You can test it via:

```
p4metrics --config p4metrics.yaml --debug --dry.run
```

Check that appropriate files are listed in your metrics dir (and are being updated every minute), e.g.

```
ls -ltr /hxlogs/metrics/
```

node exporter

Make sure **node_exporter** is working (it is easy for there to be permissions access problems to the metrics dir).

Assuming you have installed the **/etc/systemd/system/node_exporter.service** then find the **ExecStart** line and run it manually and check for errors (optionally appending "--log.level=debug").

Check that you can see values:

```
curl localhost:9100/metrics | grep p4_
```

If the above is empty, then double check the permissions on `/hxlogs/metrics` or the directory you have configured. Look for errors in:

```
sudo journalctl -u node_exporter --no-pager | less
```

prometheus

Access page <http://localhost:9090> in your browser and search for some metrics.

Grafana

Check that a suitable data source is setup (i.e. Victoria Metrics or Prometheus - see below)

Use the **Explore** option to look for some basic metrics, e.g. just start typing `p4_` and it should autocomplete if it has found `p4_` metrics being collected. Alternatively try `node_time_seconds`

Advanced config options

For improved security:

- consider LDAP integration for Grafana
- implement appropriate authentication for the various end-points such as Prometheus and `node_exporter`

Windows Installation

The above instructions are all for Linux. However, all the components have Windows binaries, with the exception of the systemd timer service for `lslocks` monitoring (`monitor_wrapper.sh` and `monitor_metrics.py`). Note that `p4metrics` now has a Windows executable version (it wraps the `p4.exe` CLI)

Details:

- Grafana has a Windows Installer: [Grafana Installer](#)
- Prometheus has a Windows executable: [Prometheus Executable](#)
- Instead of Node Exporter use: [Windows Exporter](#)
- P4Prometheus has a Windows executable: [P4prometheus Executable](#)
- P4Metrics has a Windows executable: [P4metrics Executable](#)

For testing it is recommended just to run the various executables from command line first and test with Prometheus and Grafana. This allows you to test with firewalls/ports/access rights etc. When it is all working, you can wrap up and install each binary as a Service as noted below.

Windows Exporter

This used to be called "WMI Exporter".

You can download a release, e.g. https://github.com/prometheus-community/windows_exporter/releases

Note that when you run the .MSI file it may appear that it has done nothing - there are no dialogs etc! In fact it will have installed a service called `windows_exporter`. Look for it with `RegEdit.exe`

under the key `HKLM\SYSTEM\CurrentControlSet\Services`.

Edit the executable and add the similar parameter to `node_exporter`: `--collector.textfile.directories` which must be correctly set and agree with the value used by P4prometheus.

Note that the default port is 9182.

You can test in a similar way to on Linux:

```
curl http://localhost:9182/metrics
```

and see what the output is.

P4prometheus on Windows

The executable takes the `--config` parameter and the yaml file is same format as for Linux version. You can specify paths with forward slashes if desired, e.g. `c:/p4/metrics`

Running p4metrics

Starting with the version [draft version of p4metrics](#) create locally and adjust path settings, e.g. `c:/p4/metrics`

Test the tool with your installation (analyse it's settings). First make sure your admin user is logged in.

```
p4metrics.exe --config p4metrics.yaml --debug --dry.run
```

It is important that the user account used has a long login ticket specified.

Installing Programs as Services

To install as a service using for example [NSSM - Non Sucking Service Manager!](#) to wrap the Prometheus/Windows Exporter/P4Prometheus binaries downloaded above.

Note that `Windows Exporter` has an MSI installer which will install it as a service automatically (if run with Windows Admin privileges). See the [Installation section](#) for more details and options.

You may wish to use `regedit.exe` to find the installed service (it just "does it") and tweak any parameters you want to add.

Note recommendation above regarding getting things working on command line first (e.g. with debug options).